There are lots more options and features we could use,[5] but this basic docker-stack.yml file is a good starting point. However, before we can use it to deploy our app, we need to make some changes to it to set up our database.

Migrating the Database

We saw on page 78 that the Postgres image automatically creates the default database if it doesn't exist. However, currently nothing ensures that the migrations have been run; in development, we just did this manually. We need to ensure that our app has a fully migrated database when it launches.

One way you *might* think of achieving this is with the entrypoint concept we introduced on page 121 to solve the server PID issue. We already have a docker-entrypoint.sh file that is run just prior to launching our app; this might lead you to try migrating the database as follows:

```
#!/bin/sh

set -e

if [ -f tmp/pids/server.pid ]; then
  rm tmp/pids/server.pid
fi

# BAD IDEA...
bin/rails db:migrate

exec "$@"
```

Although it seems like this approach *should* work, unfortunately it doesn't scale well. Were you to start three replicas of your app at the same time, each would try to migrate the database, which can lead to locking issues that prevent the app from starting. Another downside is that you can't migrate the database *independently*; it is tied to the app launch.

A better solution is to create a separate *service* whose sole responsibility is to migrate the database. Unlike our other services which are long-lived, this would be a *one-shot container*[6]—it would execute a single command that is expected to terminate (we'll see how to declare this shortly). The nice thing about this approach is that it gives us fine-grained control to migrate our production database at any point, just by relaunching the db-migrator service. However, it also introduces a challenge.

5. https://docs.docker.com/compose/compose-file/#deploy
6. https://blog.alexellis.io/containers-on-swarm/